

Functions

```
In [1]: def fun():  
        print('good morning ')
```

```
In [2]: fun()
```

good morning

```
In [3]: def fun():  
        print('good morning ')  
        fun()
```

good morning

```
In [4]: def greet():  
        print('hello')  
        print('good morning')
```

```
In [5]: def greet():  
        print('hello')  
        print('good morning')  
        greet()
```

hello
good morning

```
In [6]: def greet():  
        print('hello')  
        print('good morning')  
        greet()  
  
        def greet():  
            print('hello')  
            print('good morning')  
            greet()  
  
        def greet():  
            print('hello')  
            print('good morning')  
            greet()
```

hello
good morning
hello
good morning
hello
good morning

```
In [7]: def greet():  
        print('hello good morning boss')
```

```
greet()
```

```
hello good morning boss
```

```
In [42]: def greet():
          print('hello good morning boss')

          greet()

          greet()

          greet()

          greet()
```

```
hello good morning boss
hello good morning boss
hello good morning boss
hello good morning boss
```

- function are always reuseable
- in one code you can write multiple function as well
- FUNCTION ARGUMENT
- How to pass parameter to a function & what happend to the variable when you pass to a function & if you modify the value then what happen
- every code check with debug

```
In [9]: def add(x,y):
          c = x+y
          print(c)

          add(5,6,7,8)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 5
      2     c = x+y
      3     print(c)
----> 5 add(5,6,7,8)

TypeError: add() takes 2 positional arguments but 4 were given
```

```
In [10]: def add(x,y):
           c = x+y
           print(c)

           add(5,6)
```

11

```
In [11]: def add(x,y,z):
          c=x+y+z+m
          print(c)
          add(1,4,5)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[11], line 4
      2     c=x+y+z+m
      3     print(c)
----> 4 add(1,4,5)

Cell In[11], line 2, in add(x, y, z)
      1 def add(x,y,z):
----> 2     c=x+y+z+m
      3     print(c)

NameError: name 'm' is not defined
```

```
In [12]: def add(x,y,z,m):
          c=x+y+z+m
          print(c)
          add(1,4,5,6)
```

16

```
In [13]: def greet():
          print('hello')
          print('good morning')
          greet()
```

hello
good morning

```
In [14]: def add(x,y):
          c = x+y
          print(c)
          add(5,6)
```

11

```
In [15]: def greet():
          print('hello')
          print('good morning')
          greet()

          def add(x,y):
              c = x+y
              print(c)
              add(5,6)
```

hello
good morning
11

```
In [16]: def greet():
          print('hello')
```

```
print('good morning')
def add(x,y):
    c = x+y
    print(c)

add(5,6)
greet()
```

```
11
hello
good morning
```

```
In [17]: def greet():
          print('hello')
          print('good noon')

          def add(x,y):
              c=x+y
              print(c)

          def sub(x,y):
              d = x-y
              print(d)

          greet()
          add(5,4)
          sub(10,2)
```

```
hello
good noon
9
8
```

```
In [18]: def add_sub(x,y):
          c = x+y
          d = x-y
          print(c)
          print(d)

          add_sub(10,5)
```

```
15
5
```

```
In [19]: def add_sub(x,y):
          c = x+y
          d = x-y
          return c,d

          add_sub(10,5)
```

```
Out[19]: (15, 5)
```

```
In [20]: def add_sub(x,y):
          c= x+y
          d= x-y
          return c, d
```



```
result = add_sub(5,4)

print(result)
```

(9, 1)

```
In [93]: def add_sub(x,y):
          c= x+y
          d= x-y
          return c, d

          result1,result2 = add_sub(5,4)

          print(result1,result2)
```

9 1

formal argument & Actual argument

```
In [43]: def add(x,y):
          c = x+y
          print(c)
          add(5,6)
```

11

Actual Arguments

- position
- keyword
- default
- variable length

Positional argument

```
In [91]: def person(name,age):
          print(name)
          print(age)

          person('nit',23,34)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[91], line 5
      2     print(name)
      3     print(age)
----> 5 person('nit',23,34)

TypeError: person() takes 2 positional arguments but 3 were given
```

default argument

```
In [24]: def person(name,age):  
         print(name)  
         print(age)  
  
         person('nit',23)
```

```
nit  
23
```

```
In [25]: def person(name,age):  
         print(name)  
         print(age)  
  
         person(23,'nit')
```

```
23  
nit
```

```
In [26]: def person(name,age):  
         print(name)  
         print(age+1)  
  
         person(23,'nit')
```

```
23
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[26], line 5  
      2     print(name)  
      3     print(age+1)  
----> 5 person(23,'nit')
```

```
Cell In[26], line 3, in person(name, age)  
      1 def person(name,age):  
      2     print(name)  
----> 3     print(age+1)
```

```
TypeError: can only concatenate str (not "int") to str
```

keyword argument

```
In [27]: def person(name,age):  
         print(name)  
         print(age+1)  
  
         person(age=23, name='nit')
```

```
nit  
24
```

```
In [28]: def person(name,age):
          print(name)
          print(age+1)

          person(age1=23, name='nit')
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[28], line 5
      2     print(name)
      3     print(age+1)
----> 5 person(age1=23, name='nit')

TypeError: person() got an unexpected keyword argument 'age1'. Did you mean 'age'?
```

```
In [29]: def person(name,age1):
          print(name)
          print(age1+1)

          person(age1=23, name='nit')
```

```
nit
24
```

```
In [30]: def person(name,age,city):
          print(name)
          print(age+1)
          print(city)

          person(age=23, name='nit', city = 'hyd')
```

```
nit
24
hyd
```

```
In [31]: def person(name, age=18):
          print(name)
          print(age)

          person('nit', 24)
```

```
nit
24
```

Python Functions Without Arguments

```
In [33]: def greet():
          print("Hello! Welcome to Python.")
```

```
In [34]: greet()
```

```
Hello! Welcome to Python.
```

```
In [36]: def goodbye():
          print("Goodbye! Have a nice day.")
```

```
goodbye()
```

Goodbye! Have a nice day.

```
In [39]: def show_date():  
         from datetime import date  
         print("Today's date is:", date.today())  
         show_date()
```

Today's date is: 2025-12-01

```
In [40]: def welcome_message():  
         print("Welcome to the Python programming course!")  
         welcome_message()
```

Welcome to the Python programming course!

```
In [41]: def motivation():  
         print("Keep going! You're doing great.")  
         motivation()
```

Keep going! You're doing great.

```
In [42]: def calculator_instructions():  
         print("Enter two numbers and an operator (+, -, *, /).")  
         calculator_instructions()
```

Enter two numbers and an operator (+, -, *, /).

```
In [43]: def course_info():  
         print("Course: Python Basics\nDuration: 4 weeks")  
         course_info()
```

Course: Python Basics
Duration: 4 weeks

```
In [44]: def morning_greeting():  
         print("Good morning! Start your day with positivity.")  
         morning_greeting()
```

Good morning! Start your day with positivity.

```
In [45]: def ascii_heart():  
         print("<3 <3 <3")  
         ascii_heart()
```

<3 <3 <3

```
In [48]: def reminder():  
         print("Remember to take breaks while coding!")  
         reminder()
```

Remember to take breaks while coding!

Python Functions WITH Arguments

```
In [49]: def add(a, b):  
         return a + b
```

```
add(5,8)
```

Out[49]: 13

```
In [52]: def square(num):  
         return num * num  
         square(6)
```

Out[52]: 36

```
In [53]: def area_rect(length, width):  
         return length * width  
         area_rect(6,8)
```

Out[53]: 48

```
In [54]: def is_eligible(age):  
         return age >= 18  
         is_eligible(6)
```

Out[54]: False

```
In [55]: def is_eligible(age):  
         return age >= 18  
         is_eligible(19)
```

Out[55]: True

```
In [56]: def simple_interest(p, r, t):  
         return (p * r * t) / 100  
         simple_interest(5,2,8)
```

Out[56]: 0.8

```
In [57]: def simple_interest(p, r, t):  
         return (p * r * t) / 100  
         simple_interest(10,9,100)
```

Out[57]: 90.0

```
In [59]: def repeat(message, times):  
         for _ in range(times):  
             print(message)  
         repeat('HI',15)
```

HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI
HI

```
In [60]: def bigger(a, b):  
         if a > b:  
             return a  
         return b  
         bigger(8,9)
```

Out[60]: 9

```
In [61]: def power(base, exponent):  
         return base ** exponent  
         power(6,9)
```

Out[61]: 10077696

```
In [62]: def is_divisible(a, b):  
         return a % b == 0  
         is_divisible(2,7)
```

Out[62]: False

```
In [63]: def average(a, b, c):  
         return (a + b + c) / 3  
         average(2,7,19)
```

Out[63]: 9.333333333333334

Functions With Default Arguments

```
In [66]: def greet(name="Guest"):  
         print("Hello,", name)  
         greet()
```

Hello, Guest

```
In [67]: def greet(name="Guest"):  
         print("Hello,", name)  
         greet("likihth")
```

Hello, likihth

```
In [68]: def power(a, b=2):  
         return a ** b  
         power(2)
```

Out[68]: 4

```
In [70]: def repeat(msg, times=1):  
         for _ in range(times):  
             print(msg)  
         repeat('hi likhith')
```

hi likhith

```
In [71]: def repeat(msg, times=1):  
         for _ in range(times):  
             print(msg)  
         repeat('hi likhith',10)
```

hi likhith
hi likhith
hi likhith
hi likhith
hi likhith
hi likhith
hi likhith
hi likhith
hi likhith
hi likhith
hi likhith

```
In [72]: def area(length, width=1):  
         return length * width  
         area(10)
```

Out[72]: 10

```
In [73]: def area(length, width=1):  
         return length * width  
         area(10,6)
```

Out[73]: 60

```
In [74]: def price_after_discount(price, discount=10):  
         return price - (price * discount / 100)  
         price_after_discount(1000)
```

Out[74]: 900.0

```
In [75]: def price_after_discount(price, discount=10):  
         return price - (price * discount / 100)  
         price_after_discount(1000,15)
```

Out[75]: 850.0

```
In [76]: def welcome(message="Good Morning"):
         print(message, "User!")
         welcome()
```

Good Morning User!

```
In [81]: def welcome(message="Good Morning"):
         print(message, "User!")
         welcome("likhith")
```

likhith User!

```
In [82]: def convert_temp(celsius, unit="F"):
         if unit == "F":
             return (celsius * 9/5) + 32
         return celsius + 273
         convert_temp(8)
```

Out[82]: 46.4

```
In [83]: def convert_temp(celsius, unit="F"):
         if unit == "F":
             return (celsius * 9/5) + 32
         return celsius + 273
         convert_temp(8,3)
```

Out[83]: 281

```
In [84]: def student(name, grade="Not Assigned"):
         print("Name:", name)
         print("Grade:", grade)
         student("likhith")
```

Name: likhith

Grade: Not Assigned

```
In [86]: def student(name, grade="Not Assigned"):
         print("Name:", name)
         print("Grade:", grade)
         student("likhith", "A")
```

Name: likhith

Grade: A

```
In [87]: def loan_interest(amount, rate=8.5):
         return amount * rate / 100
         loan_interest(10000)
```

Out[87]: 850.0

```
In [88]: def loan_interest(amount, rate=8.5):
         return amount * rate / 100
         loan_interest(10000,8)
```

Out[88]: 800.0


```
In [90]: def show_text(text, border="*"):
          print(border * 5, text, border * 5)
          show_text("hi")

***** hi *****
```

variable length argument

```
In [1]: def sum(a, b):
          c = a+b
          return c

          sum(5,6)
```

Out[1]: 11

```
In [2]: def sum(a, b):
          c = a+b
          return c

          sum(5,6,7,8,9,10)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[2], line 5
      2 c = a+b
      3 return c
----> 5 sum(5,6,7,8,9,10)

TypeError: sum() takes 2 positional arguments but 6 were given
```

```
In [3]: def sum(a, *b):
          c = a+b
          return c

          sum(5,6,7,8,9,10)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 5
      2 c = a+b
      3 return c
----> 5 sum(5,6,7,8,9,10)

Cell In[3], line 2, in sum(a, *b)
      1 def sum(a, *b):
----> 2 c = a+b
      3 return c

TypeError: unsupported operand type(s) for +: 'int' and 'tuple'
```

```
In [4]: def sum(a, *b): # 1st argument is fixed but for 2nd argument
          #c = a+b
          print(type(a))
```

```
print(type(b))

sum(5,6,7,8)
```

```
<class 'int'>
<class 'tuple'>
```

```
In [5]: def sum(a, *b): # 1st argument is fixed & we fetch each value from the tuple & we c
        c = a

        for i in b:
            c = c + i
        print(c)

sum(5,6,7,8,9,10,100,200,300)
```

645

```
In [6]: def sum(a, *b): # 1st argument is fixed & we fetch each value from the tuple & we c
        c = a

        for i in b:
            c = c + i
        print(c)

sum(5,6,7,8)
```

26

- positional argument
- keyword argument
- default
- variable length (* at last arg) | (args)
- keyword + variable length(kwargs)

KWARGS (key worded variable length arguments)

```
In [7]: def person():
        person('ALEX', 36, 'JOHN', 987767)
```

```
In [8]: def person(name, *data):
        print(name)
        print(data)

person('ALEX', 36, 'JOHN', 987767)
```

```
ALEX
(36, 'JOHN', 987767)
```

```
In [9]: def person(name,*data):
        print('name')
```

```
print(data)

person('ALEX', age = 36, home_place = 'southcity', mob = 987767)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 5
      2 print('name')
      3 print(data)
----> 5 person('ALEX', age = 36, home_place = 'southcity', mob = 987767)

TypeError: person() got an unexpected keyword argument 'age'
```

```
In [10]: def person(name, **data):
          print('name')
          print(data)

          person('ALEX', age = 36, home_place = 'southcity', mob = 987767, edu = 'phd')
```

name
{'age': 36, 'home_place': 'southcity', 'mob': 987767, 'edu': 'phd'}

global variable vs local variable

- Sometimes we are creating variable inside the function and sometimes we are creating variable outside of the function
- when we declare variable outside of the function that concept called as scope

```
In [11]: a = 10

print(a)
```

10

```
In [12]: a = 10

def something():
    b = 15
    print('in function', b)
    print('out function', a)
```

```
In [13]: a = 10

def something():
    b = 15

    print('in function', b)
    print('out function', a)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[13], line 5
      2 def something():
      3     b = 15
----> 5 print('in function',b)
      6 print('out function',a)

NameError: name 'b' is not defined
```

```
In [14]: a = 10

def something():
    b = 15
    print('in function',b)

print('out function',a)
```

out function 10

```
In [15]: a = 10

def something():
    a = 15

print('in function',a)

print('out function',a)
```

in function 10
out function 10

```
In [16]: a = 10

def something():
    b = 15
    print('in function',b)

something()

print('out function',a)
```

in function 15
out function 10

```
In [17]: a = 10 #global var

def something():
    b = 55 # local var
    print('in function',b)
    something()

print('out function',a)
```

in function 55
out function 10

```
In [18]: # if i want to define global variabel inside the function
a = 10

def something():
    global a
    b = 15 # 15 is converted to Local when user assigned global a
    print('in function',b)
    print('gloabl variable', a)
something()
print('out function',a)
```

```
in function 15
gloabl variable 10
out function 10
```

```
In [19]: x = 10 # Global variable

def update_x():
    global x # Declare that we are using the global variable x
    x += 5 # Modify the global variable

update_x()
print(x) # Output: 15
```

```
15
```

```
In [20]: x = 10 # Global variable

def update_x():
    globals()['x'] += 5 # Access and modify the global variable

update_x()
print(x) # Output: 15
```

```
15
```

```
In [21]: import keyword
keyword.kwlist
```

```
Out[21]: ['False',
          'None',
          'True',
          'and',
          'as',
          'assert',
          'async',
          'await',
          'break',
          'class',
          'continue',
          'def',
          'del',
          'elif',
          'else',
          'except',
          'finally',
          'for',
          'from',
          'global',
          'if',
          'import',
          'in',
          'is',
          'lambda',
          'nonlocal',
          'not',
          'or',
          'pass',
          'raise',
          'return',
          'try',
          'while',
          'with',
          'yield']
```

```
In [41]: def count(lst):
          pass
          lst = [1,2,3,4,8,9,10]
          lst
```

```
Out[41]: [1, 2, 3, 4, 8, 9, 10]
```

```
In [28]: def count(lst):

          even = 0
          odd = 0

          for i in lst:
              if i%2 == 0:
                  even += 1
              else:
                  odd +=1
          return even,odd
```

```
lst = [1,2,3,4,8,9,10]
even, odd = count(lst)

print(even)
print(odd)
```

4
3

In [29]: **def** count(lst):

```
    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1
        else:
            odd +=1
    return even,odd
```

```
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11,12,13]
even,odd = count(lst)
```

```
print("Even Number: {} and odd Number : {}".format(even,odd))
#format is function belongs to string & bydefault you need to pass any parameter
```

Even Number: 6 and odd Number : 7

In [30]: *# in programmin we need to continue these process thats why we need to use loop hear*

```
def fib(n):
    a = 0
    b = 1

    print(a)
    print(b)

    for i in range(0, n):
        c = a + b
        a = b
        b = c

        print(c)

fib(10)
```

0
1
1
2
3
5
8
13
21
34
55
89

Factorial of a number in python

```
In [44]: def fact(n):  
         f = 1  
         for i in range(1, n+1):  
             f = f*i  
  
         return f  
  
x = 6  
result = fact(x)  
print(result)
```

720

```
In [45]: def fact(n):  
         f = 1  
         for i in range(1, n+1):  
             f = f*i  
  
         return f  
  
x = 5  
result = fact(x)  
print(result)
```

120

Recurrision function

- recurrison function calling itself

```
In [46]: def wish():  
         print('hello')  
         print('hi')  
         wish()
```

hello
hi


```
def wish(): print('hello') print('hi') wish() wish()
```

```
In [49]: import sys  
sys.getrecursionlimit()
```

Out[49]: 3000

```
In [52]: import sys  
sys.setrecursionlimit(200)  
print(sys.getrecursionlimit())
```

200

```
In [53]: import sys  
sys.getrecursionlimit()
```

Out[53]: 200

```
In [54]: def wish():  
          print('hello')  
  
          wish()  
  
          wish()
```

hello
hello

```
In [55]: def wish():  
          print('hello')  
          wish()  
  
          wish()
```

[illegible]

[illegible]

[illegible]

```
hello
hello
hello
hello
hello
hello
hello
```

```
-----
RecursionError                                Traceback (most recent call last)
Cell In[55], line 5
      2     print('hello')
      3     wish()
----> 5 wish()

Cell In[55], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[55], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

[... skipping similar frames: wish at line 3 (172 times)]

Cell In[55], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[55], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     wish()

File C:\ProgramData\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

RecursionError: maximum recursion depth exceeded
```

```
In [56]: import sys
         sys.setrecursionlimit(150)
         print(sys.getrecursionlimit())
```

```
i = 0

def wish():
    global i
    i += 1
    print('hello', i)
    wish()
wish()
```

```
150
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
```

```
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83
hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
```



```
hello 112  
hello 113  
hello 114  
hello 115  
hello 116  
hello 117  
hello 118  
hello 119  
hello 120  
hello 121  
hello 122  
hello 123  
hello 124  
hello 125
```

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[56], line 12
     10     print('hello', i)
     11     wish()
--> 12 wish()

Cell In[56], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()

Cell In[56], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()

[... skipping similar frames: wish at line 11 (122 times)]

Cell In[56], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()

Cell In[56], line 10, in wish()
     8 global i
     9 i += 1
--> 10 print('hello', i)
    11 wish()

File C:\ProgramData\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

RecursionError: maximum recursion depth exceeded

```

```

In [57]: i = 0

def wish():
    global i
    i += 1
    print('hello', i)
    wish()
wish()

```

```
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
```

```
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83
hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
hello 112
```

```
hello 113  
hello 114  
hello 115  
hello 116  
hello 117  
hello 118  
hello 119  
hello 120  
hello 121  
hello 122  
hello 123  
hello 124  
hello 125
```

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[57], line 8
      6     print('hello', i)
      7     wish()
----> 8 wish()

Cell In[57], line 7, in wish()
      5 i += 1
      6 print('hello', i)
----> 7 wish()

Cell In[57], line 7, in wish()
      5 i += 1
      6 print('hello', i)
----> 7 wish()

[... skipping similar frames: wish at line 7 (122 times)]

Cell In[57], line 7, in wish()
      5 i += 1
      6 print('hello', i)
----> 7 wish()

Cell In[57], line 6, in wish()
      4 global i
      5 i += 1
----> 6 print('hello', i)
      7 wish()

File C:\ProgramData\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

RecursionError: maximum recursion depth exceeded

```

```
In [58]: import sys
        sys.getrecursionlimit()
```

```
Out[58]: 150
```

```
In [59]: def wish():
        print('hello')
        wish()
        wish()
```

[illegible]

[illegible]


```
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello
```

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[59], line 4
      2     print('hello')
      3     wish()
----> 4 wish()

Cell In[59], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[59], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

[... skipping similar frames: wish at line 3 (122 times)]

Cell In[59], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[59], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     wish()

File C:\ProgramData\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

RecursionError: maximum recursion depth exceeded

```

```
In [62]: import sys
        print(sys.getrecursionlimit())
```

150

```
In [63]: sys.setrecursionlimit(1000)
```

```
In [64]: print(sys.getrecursionlimit())
```

1000

```
In [65]: import sys
sys.setrecursionlimit(150)
print(sys.getrecursionlimit())

i = 0

def wish():
    global i
    i += 1
    print('hello', i)
    wish()
wish()
```

```
150
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
```

```
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83
hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
```

```
hello 112  
hello 113  
hello 114  
hello 115  
hello 116  
hello 117  
hello 118  
hello 119  
hello 120  
hello 121  
hello 122  
hello 123  
hello 124  
hello 125
```

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[65], line 12
     10     print('hello', i)
     11     wish()
--> 12 wish()

Cell In[65], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()

Cell In[65], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()

[... skipping similar frames: wish at line 11 (122 times)]

Cell In[65], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()

Cell In[65], line 10, in wish()
     8 global i
     9 i += 1
--> 10 print('hello', i)
    11 wish()

File C:\ProgramData\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
    655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
    656     """Write to current stream after encoding if necessary
    657
    658     Returns
    659     (...)
    662
    663     """
--> 664     parent = self.parent_header
    666     if not isinstance(string, str):
    667         msg = f"write() argument must be str, not {type(string)}" # type:ignore[unreachable]

RecursionError: maximum recursion depth exceeded

```

FACTORIAL USING RECURSSION

- recursion is function calls itself

```

In [66]: def fact(n):
         if n==0:
             return 1

```

```

    return n * fact(n-1)

result = fact(5)

result

```

Out[66]: 120

Function without name is called - ANONYMOUS FUNCTION OR LAMBDA

```

In [67]: def square(a):
        return a * a

square(5)
# what if i dont want to call square() multiple times

```

Out[67]: 25

```

In [68]: def square(a):
        return a * a

result = square(5)
print(result)
# what if i dont want to call square() multiple times

```

25

```

In [69]: #Lambda expresion or lambda function
f = lambda a : a * a # hear a is an argument & operation in the argument is a * a
result = f(5)
result
# hear anonymous function is called lambda
# remember lambda always you need to assign as function cuz function are object in p

```

Out[69]: 25

```

In [70]: f = lambda a, b : a + b
        f1 = lambda a, b : a - b

result = f(1,4)
result1 = f1(4,1)

print(result)
print(result1)

```

5
3

```

In [71]: f = lambda a, b : a + b
        f1 = lambda a, b : a - b
        f2 = lambda a,b : a * b

result = f(1,4)
result1 = f1(4,1)
result2 = f2(4,1)

```



```
print(result)
print(result1)
print(result2)
```

5
3
4

In [72]: *#Lets define function which will add 2 number*
we are defining a function whcih doesnt have name
#lambda expresion or lambda function

```
f = lambda a, b : a + b
f1 = lambda a, b : a - b

result = f(1,4)
result1 = f1(2, 3)

print(result)
print(result1)
```

5
-1

How can we use lambda in other function like - filter, map & reduce

In [73]: *#Lets take one list & i want to find the list of even numbers*
nums = [3,2,6,8,4,6,2,9]
evens = list(filter(is_even, nums)) #is_even is not an inbuild function

```
-----
NameError                                Traceback (most recent call last)
Cell In[73], line 3
      1 #lets take one list & i want to find the list of even numbers
      2 nums = [3,2,6,8,4,6,2,9]
----> 3 evens = list(filter(is_even, nums))

NameError: name 'is_even' is not defined
```

In [74]: **def** is_even(n):
 return n % 2 == 0

```
nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
print(evens)
```

[2, 6, 8, 4, 6, 2]

In [75]: **def** is_odd(n):
 return n % 2 != 0

```
nums = [3,2,6,8,4,6,2,9]

odd = list(filter(is_odd, nums))
print(odd)
```

[3, 9]

```
In [76]: # Lets write above function using help of Lambda & Lambda helps to reduce the Line
nums = [3,2,6,8,4,6,2,9]

evens = list(filter(lambda n : n%2 == 0, nums))

print(evens)
```

[2, 6, 8, 4, 6, 2]

```
In [77]: nums = [3,2,6,8,4,6,2,9]

odd = list(filter(lambda n : n%2 !=0, nums))

print(odd)
```

[3, 9]

```
In [79]: # Lets write above function using help of Lambda & Lambda helps to reduce the Line
nums = [3,2,6,8,4,6,2,9, 34,77, 120]

evens = list(filter(lambda n : n%2 ==0, nums))
odd = list(filter(lambda n : n%2 !=0, nums))

print(evens)
print(odd)
```

[2, 6, 8, 4, 6, 2, 34, 120]

[3, 9, 77]

- What ever even number I have from the assigned list
- I want to double the even number i.e 2 become 4 || 4 become 6 || 6 become 8
- that we will do using map function
- this largely we are using in google map reduce programme
- we can build using user define & lambda

```
In [80]: def update(n):
        return n+2

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(update, evens))

print(evens)
print(double)
```

[2, 6, 8, 4, 6, 2]

[4, 8, 10, 6, 8, 4]

```
In [81]: nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
```

```
double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n+2, evens))
doubble_1 = list(map(lambda n : n-2, evens))

print(evens)
print(double)
print(double_)
print(doubble_1)
```

```
[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
[4, 8, 10, 6, 8, 4]
[0, 4, 6, 2, 4, 0]
```

```
In [82]: nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n-2, evens))

print(double)
print(double_)
```

```
[4, 12, 16, 8, 12, 4]
[0, 4, 6, 2, 4, 0]
```

```
In [83]: nums = [3,2,6,8,4,6,2,9]
evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n-2, evens))
double1 = list(map(lambda n : n+2, evens))

print(double)
print(double_)
print(double1)
```

```
[4, 12, 16, 8, 12, 4]
[0, 4, 6, 2, 4, 0]
[4, 8, 10, 6, 8, 4]
```

- i want to perform reduce now
- i want reduce all the values
- reduce you can add only 2 values
- [4, 12, 16, 8, 12, 4] if you sum everything then you will get 56

```
In [84]: from functools import reduce

def add_all(a,b):
    return a+b

nums = [3,2,6,8,4,6,2]

evens = list(filter(is_even, nums))
```

```
double = list(map(lambda n : n*2, evens))

sums = reduce(add_all, double)
sums
print(sums)
```

56

```
In [85]: a = [7,8]
print(type(a))
```

<class 'list'>

```
In [86]: from functools import reduce

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
sums = (reduce(lambda a,b : a + b, double))

print(evens)
print(double)
print(sums)
```

[2, 6, 8, 4, 6, 2]

[4, 12, 16, 8, 12, 4]

56

python decorators

- This is one of amazing feature in python
- We know that functions are built on certain tasks
- Let's understand we have some predefined functions
- div() function which takes 2 parameters

```
In [2]: def div(a,b):
        print(a / b)
        div(4,2)
# but what if we pass the value 2, 4
```

2.0

```
In [3]: def div(a, b):
        print(a / b)
        div(2,4)
```

0.5

- I want different logic here
- It does not matter in which sequence I pass the value always numerator should be greater than denominator (numerator > denominator)
- In the above function when I divide 2/4 (numerator < denominator)

- even though user pass 2/4 but inbuilt it should consider as 4 / 2 but how to swap lets discuss
- before division 2/4 lets apply the logic & logic is if $2 < 4$ then i want to swap

```
In [4]: def div(a,b):

        if a<b:
            a,b = b,a
        print(a / b)

div(2,4)
```

2.0

- lets understand the twist, imagine the code you dont have , you work in the organization
- inbuilt the code & you dont want to change the existing code
- without impacting on existing code you need to get the solution
- being a developer you need to swap the value without touching the existing function
- thats the reason DECORATOR comes into the picture
- using the help of DECORATOR you can add the extra feature in the existing function
- lets create a new function for decorator
- we can write functin inside the funtion thats why we called as python is functional programming language

```
In [5]: # using help of the decorator you can add the extra feature in the exicting functio

def div(a,b):
    print(a / b)

def div_decorator(func): # hear div_dectorator will accept the div function
    def inner(a,b):
        if a<b:
            a,b = b,a
        return func(a,b)
    return inner

div = div_decorator(div)

div(2,4)
```

2.0

```
In [6]: def my_decorator(func):
        def wrapper():
            print("Something is happening before the function is called.")
            #func()
            print("Something is happening after the function is called.")
        return wrapper

        @my_decorator
```

```
def say_hello():  
    print("Hello!")  
  
say_hello()
```

Something is happening before the function is called.
Something is happening after the function is called.

```
In [7]: def my_decorator(func):  
        def wrapper():  
            print("Something is happening before the function is called.")  
            func()  
            print("Something is happening after the function is called.")  
        return wrapper  
  
@my_decorator  
def say_hello():  
    print("Hello!")  
  
say_hello()
```

Something is happening before the function is called.
Hello!
Something is happening after the function is called.

- the best part of the decorator is you can change the behaviour of existing function at the time of compiler
- what we done now we take new function which take func as an parameter
- you cant do like this any programming language
- we can defin the function with in the function which actually replacing using swap
- without changing existing code with the help of decorator we are get the output

Modules

- concept of debugging means - remov the bugs
- what is codeing - coding will create bugs or add the bugs
- Brekdown complex codes or logical part to different modules

```
In [ ]:
```